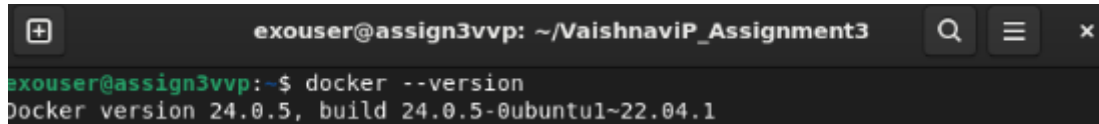


ECC Assignment - 3

Docker Installation:

1. I have created an instance in Jetstream.
2. The jetstream used has preinstalled docker in the root user.
3. Docker version 24.0.5 is being used, as observed below.



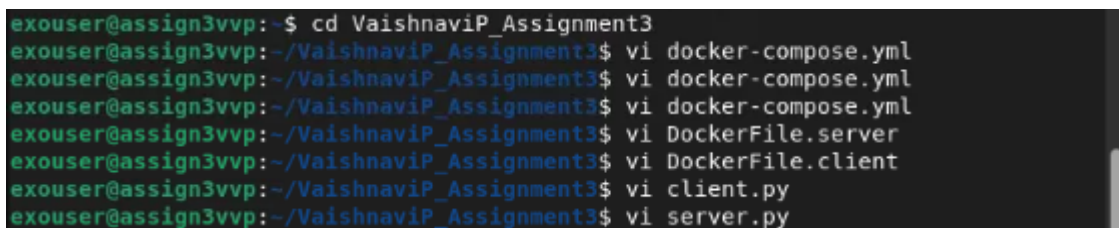
```
exouser@assign3vvp: ~/VaishnaviP_Assignment3
exouser@assign3vvp:~$ docker --version
Docker version 24.0.5, build 24.0.5-0ubuntu1~22.04.1
```

Important Note: I used the sudo commands to access volumes because the docker was already installed as the root user.

Structure of the Directory:

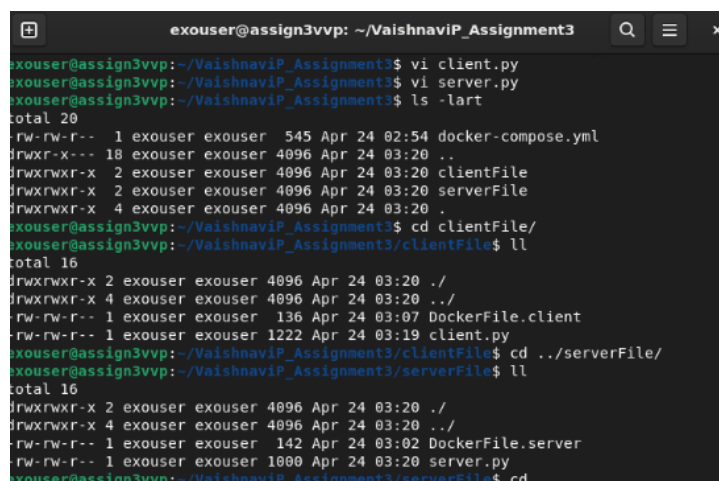
The Directory which was created to perform this assignment is as follows:

1. I have made all the required files and put them in the right order for easy access.



```
exouser@assign3vvp:~$ cd VaishnaviP_Assignment3
exouser@assign3vvp:~/VaishnaviP_Assignment3$ vi docker-compose.yml
exouser@assign3vvp:~/VaishnaviP_Assignment3$ vi docker-compose.yml
exouser@assign3vvp:~/VaishnaviP_Assignment3$ vi docker-compose.yml
exouser@assign3vvp:~/VaishnaviP_Assignment3$ vi DockerFile.server
exouser@assign3vvp:~/VaishnaviP_Assignment3$ vi DockerFile.client
exouser@assign3vvp:~/VaishnaviP_Assignment3$ vi client.py
exouser@assign3vvp:~/VaishnaviP_Assignment3$ vi server.py
```

2. I have created a folder named VaishnaviP_Assignment3, which contains three files, docker-compose.yml, clientFile, and serverFile
3. The clientFile holds two files in it, a DockerFile.client and client.py
4. The serverFile holds two files in it, a DockerFile.server and server.py
5. The screenshot that is presented below shows every file that has been produced along with their location.

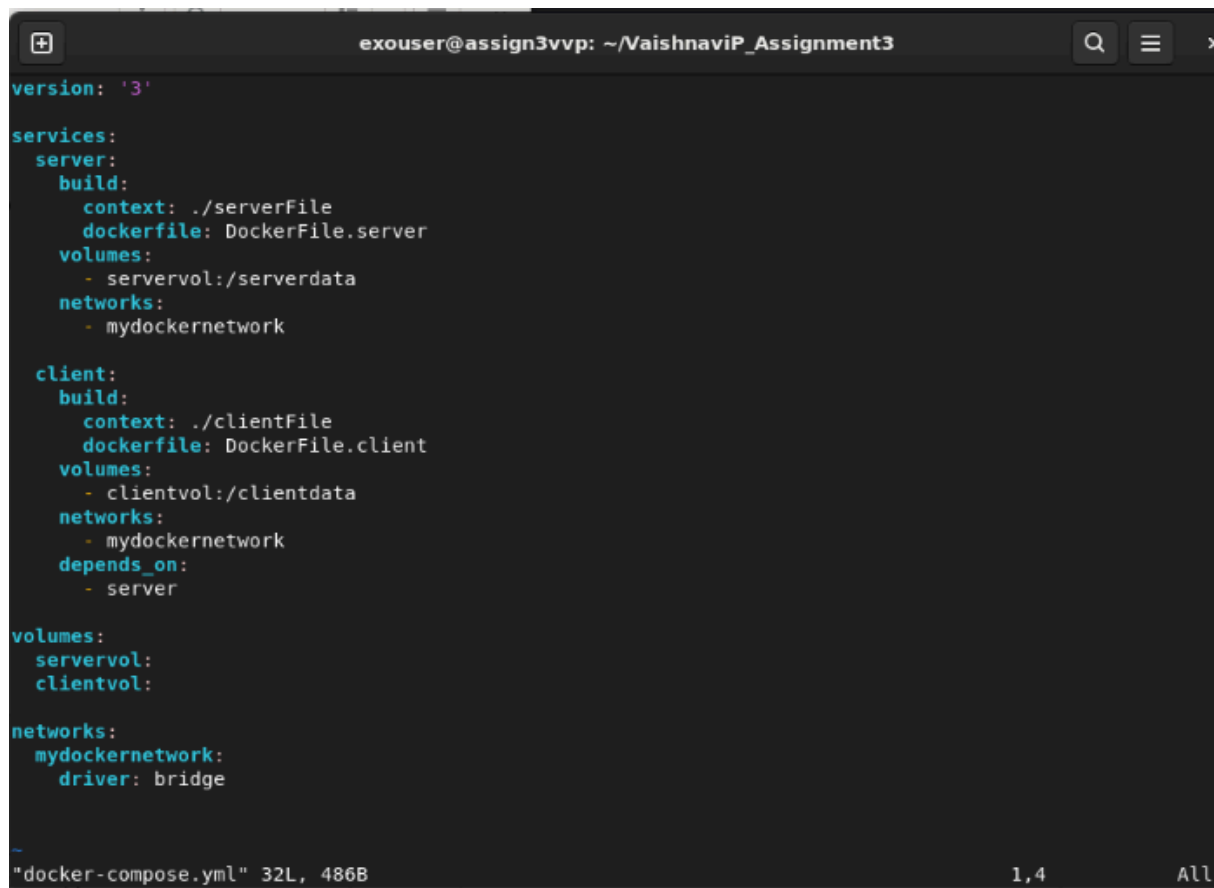


```
exouser@assign3vvp:~/VaishnaviP_Assignment3$ vi client.py
exouser@assign3vvp:~/VaishnaviP_Assignment3$ vi server.py
exouser@assign3vvp:~/VaishnaviP_Assignment3$ ls -lart
total 20
-rw-rw-r-- 1 exouser exouser 545 Apr 24 02:54 docker-compose.yml
drwxr-x-- 18 exouser exouser 4096 Apr 24 03:20 ..
drwxrwxr-x 2 exouser exouser 4096 Apr 24 03:20 clientFile
drwxrwxr-x 2 exouser exouser 4096 Apr 24 03:20 serverFile
drwxrwxr-x 4 exouser exouser 4096 Apr 24 03:20 .
exouser@assign3vvp:~/VaishnaviP_Assignment3$ cd clientFile/
exouser@assign3vvp:~/VaishnaviP_Assignment3/clientFile$ ll
total 16
drwxrwxr-x 2 exouser exouser 4096 Apr 24 03:20 ../
drwxrwxr-x 4 exouser exouser 4096 Apr 24 03:20 ../
-rw-rw-r-- 1 exouser exouser 136 Apr 24 03:07 DockerFile.client
-rw-rw-r-- 1 exouser exouser 1222 Apr 24 03:19 client.py
exouser@assign3vvp:~/VaishnaviP_Assignment3/clientFile$ cd ../serverFile/
exouser@assign3vvp:~/VaishnaviP_Assignment3/serverFile$ ll
total 16
drwxrwxr-x 2 exouser exouser 4096 Apr 24 03:20 ../
drwxrwxr-x 4 exouser exouser 4096 Apr 24 03:20 ../
-rw-rw-r-- 1 exouser exouser 142 Apr 24 03:02 DockerFile.server
-rw-rw-r-- 1 exouser exouser 1000 Apr 24 03:20 server.py
exouser@assign3vvp:~/VaishnaviP_Assignment3/serverFile$ cd
```

Files:

1. docker-compose.yml:

- I have included the code for the docker-compose.yml file.
- It uses version 3, and there are two services defined: 'server' and 'client'.
- Both of these services are connected to mydockernetwork, which is then prefixed with my project name (vaishnavip_assignment3), which is also the directory name by default.
- As stated in the assignment, each server uses named volumes (servervol and clientvol).
- Volumes save data for both "server" and "client" services in the /serverdata and /clientdata directories.
- I intended to utilize the built-in bridge network driver here.
- To ensure the correct service startup sequence, the "client" service depends on the "server" service.



```
version: '3'

services:
  server:
    build:
      context: ./serverFile
      dockerfile: DockerFile.server
    volumes:
      - servervol:/serverdata
    networks:
      - mydockernetwork

  client:
    build:
      context: ./clientFile
      dockerfile: DockerFile.client
    volumes:
      - clientvol:/clientdata
    networks:
      - mydockernetwork
    depends_on:
      - server

volumes:
  servervol:
  clientvol:

networks:
  mydockernetwork:
    driver: bridge
```

"docker-compose.yml" 32L, 486B 1,4 All

2. DockerFile.server

- The working directory is defined as /app, a volume is specified at /serverdata, and a Python 3.10 environment is set up using this Dockerfile.
- It downloads the Flask library and copies a Python script called server.py that uses Flask into the container's /app directory.
- It sets the default command to launch the Flask server on port 8080 when the container starts, as was indicated in the assignment.

```
exouser@assign3vvp: ~/VaishnaviP_Assignment3/serverFile
FROM python:3.10
WORKDIR /app
VOLUME /serverdata
COPY server.py /app/server.py
RUN pip install Flask
CMD ["python", "server.py", "8080"]
```

3. server.py

- Using a single route ("/"), it builds a Flask web application that generates a random file (GeneratedFile.txt).
- It then computes the file's SHA-256 checksum and accepts the file by including the checksum in the response headers.
- On the designated port 8080, the server listens.
- I have created various functions like send_Randomfile, create_Randomfile, calChecksum, and send_fileAsResponse, which perform the task as named.

```
exouser@assign3vvp: ~/VaishnaviP_Assignment3/serverFile
from flask import Flask, send_file, make_response
import os
import hashlib
import random
import string
import sys

app = Flask(__name__)

@app.route('/')
def send_Randomfile():
    pathOfFile = '/serverdata/GeneratedFile.txt'
    checksum = create_Randomfile(pathOfFile)
    return send_fileAsResponse(pathOfFile, 'GeneratedFile.txt', checksum)

def create_Randomfile(pathOfFile):
    #Generate a file with random content and calculate its SHA-256 checksum.
    chars = string.ascii_letters + string.digits
    contentOfFile = ''.join(random.choice(chars) for _ in range(1024))
    with open(pathOfFile, 'w') as file:
        file.write(contentOfFile)

    return calChecksum(contentOfFile)

def calChecksum(data):
    #Calculate and return the SHA-256 checksum of the provided data.
    hasher256 = hashlib.sha256(data.encode('utf-8'))
    return hasher256.hexdigest()

def send_fileAsResponse(pathOfFile, Filename, checksum):
    #Send the file with a checksum header.
    response = make_response(send_file(pathOfFile, as_attachment=True, download_name='GeneratedFile.txt', mimetype='application/octet-stream'))
    response.headers['Checksum'] = checksum
    return response

if __name__ == '__main__':
    Serverport = int(sys.argv[1]) if len(sys.argv) > 1 else 8080
    app.run(host='0.0.0.0', port=Serverport)
```

4. DockerFile.client

- This Dockerfile creates a Python 3.10 environment for a client application.
- Furthermore, it installs the Requests library and copies a Python script (client.py) to the container's /app directory.
- A volume is specified at /clientdata.



```
exouser@assign3vvp: ~/VaishnaviP_Assignment3/clientFile
FROM python:3.10
WORKDIR /app
RUN pip install requests
COPY client.py /app/client.py
VOLUME /clientdata
CMD ["python", "client.py"]
```

5. client.py

- The client program retrieves a file with a checksum from the particular server URL (http://server:8080/).
- The file is saved to /clientdata/FileReceived.txt and its checksum is confirmed with SHA-256.
- If the checksum validation fails, an error message is printed.
- I have created various functions like fetchFile_from_Server, computeSHA256_Checksum, and verifyChecksum which perform the task as named.
- While working on running both containers, I was having trouble making them run. So, to determine the source of the problem, I added quite a few print statements that informed me where it went wrong.
- The command used to verify the logs was the **docker-compose logs client**.
- To keep the system up and running once the task is completed, I included a try statement that sets the duration of the sleep log.

```
exouser@assign3vvp: ~/VaishnaviP_Assignment3/clientFile

import requests
import time
import hashlib

URL_OF_THESERVER = "http://server:8080/"
PATH_OF_FILE = '/clientdata/FileReceived.txt'

def fetchFile_from_Server(url):
    try:
        response = requests.get(url)
        if response.status_code == 200:
            with open(PATH_OF_FILE, 'wb') as f:
                f.write(response.content)
            return PATH_OF_FILE, response.headers.get('Checksum')
        else:
            response.raise_for_status()
    except requests.RequestException as e:
        print(f"Failed to download the file: {e}")
        return None, None

def computeSHA256_Checksum(Filepath):
    #Computes the SHA-256 checksum of a file.
    hasher256 = hashlib.sha256()
    with open(Filepath, 'rb') as file:
        while chunk := file.read(8192):
            hasher256.update(chunk)
    return hasher256.hexdigest()

def verifyChecksum(pathOfFile, ChecksumReceived):
    #Verifies that a given file's checksum matches the expected_checksum.
    calculatedChecksum = computeSHA256_Checksum(pathOfFile)
    return calculatedChecksum == ChecksumReceived

def main():
    print("Starting file download")
    fileDownloaded, ChecksumReceived = fetchFile_from_Server(URL_OF_THESERVER)

    if fileDownloaded is None or ChecksumReceived is None:
        print("Failed to download file or checksum is missing.")
        return # Exit if failed to download

    print(f"File downloaded: {fileDownloaded}")
    print(f"Received Checksum: {ChecksumReceived}")

    is_checksum_valid = verifyChecksum(fileDownloaded, ChecksumReceived)
    if is_checksum_valid:
        print("Success: File received and verified successfully!")
    else:
        print("Error: Checksum does not match.")

    try:
        while True:
            time.sleep(10)
    except KeyboardInterrupt:
        print("Program terminated.")

if __name__ == '__main__':
    main()
```

Working:

- I initially checked the Docker version to ensure that the most recent version (24.0.5) was used.
- As the **docker-compose build** command was not installed, I installed it using the **sudo apt install docker-compose** command.

```

exouser@assign3vvp: ~/VaishnaviP_Assignment3
exouser@assign3vvp:~/VaishnaviP_Assignment3$ docker --version
Docker version 24.0.5, build 24.0.5-0ubuntu1-22.04.1
exouser@assign3vvp:~/VaishnaviP_Assignment3$ docker-compose build
Command 'docker-compose' not found, but can be installed with:
sudo snap install docker          # version 24.0.5, or
sudo apt install docker-compose  # version 1.29.2-1
See 'snap info docker' for additional versions.
exouser@assign3vvp:~/VaishnaviP_Assignment3$ sudo apt install docker-compose
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libwpe-1.0-1 libwpebackend-fdo-1.0-1
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  python3-docker python3-dockerpty python3-dockerpty python3-dotenv
  python3-texttable python3-websocket
The following NEW packages will be installed:
  docker-compose python3-docker python3-dockerpty python3-dockerpty
  python3-dotenv python3-texttable python3-websocket
0 upgraded, 7 newly installed, 0 to remove and 3 not upgraded.
Need to get 290 kB of archives.
After this operation, 1545 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://nova.clouds.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-websocket all 1.2.3-1 [34.7 kB]
Get:2 http://nova.clouds.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-docker all 5.0.3-1 [89.3 kB]
Get:3 http://nova.clouds.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-dockerpty all 0.4.1-2 [11.1 kB]
Get:4 http://nova.clouds.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-dockerpty all 0.6.2-4 [26.9 kB]
Get:5 http://nova.clouds.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-dotenv all 0.19.2-1 [20.5 kB]
Get:6 http://nova.clouds.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-texttable all 1.6.4-1 [11.4 kB]
Get:7 http://nova.clouds.archive.ubuntu.com/ubuntu jammy/universe amd64 docker-compose all 1.29.2-1 [95.8 kB]
Fetched 290 kB in 1s (392 kB/s)
Selecting previously unselected package python3-websocket.
(Reading database ... 206147 files and directories currently installed.)
Preparing to unpack .../0-python3-websocket_1.2.3-1_all.deb ...
Unpacking python3-websocket (1.2.3-1) ...
Selecting previously unselected package python3-docker.
Preparing to unpack .../1-python3-docker_5.0.3-1_all.deb ...
Unpacking python3-docker (5.0.3-1) ...
Selecting previously unselected package python3-dockerpty.
Preparing to unpack .../2-python3-dockerpty_0.4.1-2_all.deb ...
Unpacking python3-dockerpty (0.4.1-2) ...
Selecting previously unselected package python3-dockerpty.
Preparing to unpack .../3-python3-dockerpty_0.6.2-4_all.deb ...
Unpacking python3-dockerpty (0.6.2-4) ...
Selecting previously unselected package python3-dotenv.
Preparing to unpack .../4-python3-dotenv_0.19.2-1_all.deb ...
Unpacking python3-dotenv (0.19.2-1) ...

```

- I also upgraded the pip to its latest version (24.0), using the below statement
pip install - -upgrade pip.

```

exouser@assign3vvp:~/VaishnaviP_Assignment3$ pip install --upgrade pip
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pip in /usr/lib/python3/dist-packages (22.0.2)
Collecting pip
  Downloading pip-24.0-py3-none-any.whl (2.1 MB)
    

2.1/2.1 MB 11.6 MB/s eta 0:00:00


Installing collected packages: pip
  WARNING: The scripts pip, pip3 and pip3.10 are installed in '/home/exouser/.local/bin' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed pip-24.0

```

Build and Run your server and client container.

- ★ I used docker-compose.yml to create and configure many containers.
- ★ I used **docker-compose build** to build and operate my containers, **vaishnavip_assignment3_client_1**, and **vaishnavip_assignment3_server_1**.
- ★ The **docker-compose build** command is used to create or rebuild the docker images specified in the docker-compose file.
- ★ When I execute this command in the directory containing my docker-compose.yml file, it scans the configuration and creates images for all of the services specified in the file.
- ★ Hence, successfully build the server and client containers.

```
exouser@assign3vvp: ~/VaishnaviP_Assignment3
exouser@assign3vvp:~/VaishnaviP_Assignment3$ docker-compose build
Building server
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.096kB
Step 1/6 : FROM python:3.10
--> 08678661b65f
Step 2/6 : WORKDIR /app
--> Using cache
--> 6bbf491a7365
Step 3/6 : VOLUME /serverdata
--> Using cache
--> 7c03431fd11a
Step 4/6 : COPY server.py /app/server.py
--> Using cache
--> 57d814a1fffe
Step 5/6 : RUN pip install Flask
--> Using cache
--> e20acfe112bf
Step 6/6 : CMD ["python", "server.py", "8080"]
--> Using cache
--> 2f8e8ea38939
Successfully built 2f8e8ea38939
Successfully tagged vaishnavip_assignment3_server:latest
Building client
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.608kB
Step 1/6 : FROM python:3.10
--> 08678661b65f
Step 2/6 : WORKDIR /app
--> Using cache
--> 6bbf491a7365
Step 3/6 : RUN pip install requests
--> Using cache
--> 84c341c26731
Step 4/6 : COPY client.py /app/client.py
--> Using cache
--> 4fe6231b3096
Step 5/6 : VOLUME /clientdata
--> Using cache
--> 9ea3b82a4820
Step 6/6 : CMD ["python", "client.py"]
--> Using cache
--> 603448fc3d12
Successfully built 603448fc3d12
Successfully tagged vaishnavip_assignment3_client:latest
```

- ★ I used the command **docker-compose up -d** to start the services specified in the docker-compose file.
- ★ I used the -d flag in detached mode to run services in the background.
- ★ Here, `docker-compose up -d` creates a network called **vaishnavip_assignment3_mydockernetwork**, built with default driver 'bridge' with two volumes, **vaishnavip_assignment3_servervol**, and **vaishnavip_assignment3_clientvol**, as well as two services, **vaishnavip_assignment3_server_1** and **vaishnavip_assignment3_client_1**.

Note: I am including two screenshots because, while working on the errors, I had already formed a network with the volumes. So, when the main network was created, the volumes already existed.

```
exouser@assign3vvp:~/VaishnaviP_Assignment3$ docker-compose up -d
Creating volume "vaishnavip_assignment3_servervol" with default driver
Creating volume "vaishnavip_assignment3_clientvol" with default driver
Starting vaishnavip_assignment3_server_1 ... done
Starting vaishnavip_assignment3_client_1 ... done
exouser@assign3vvp:~/VaishnaviP_Assignment3$
```

```
exouser@assign3vvp:~/VaishnaviP_Assignment3$ docker-compose up -d
Creating network "vaishnavip_assignment3_mydockernetwork" with driver "bridge"
Creating vaishnavip_assignment3_server_1 ... done
Creating vaishnavip_assignment3_client_1 ... done
```

- ★ Using the **docker-compose ps** command, we can see both the server and client containers are up and running.
- ★ The server uses the mentioned port 8080.

```
exouser@assign3vvp: ~/VaishnaviP_Assignment3
exouser@assign3vvp:~/VaishnaviP_Assignment3$ docker-compose ps
```

Name	Command	State	Ports
vaishnavip_assignment3_client_1	python client.py	Up	
vaishnavip_assignment3_server_1	python server.py 8080	Up	

- ★ After running the containers, I checked whether the volumes were successfully created using **docker volume ls**, which lists all the volumes.
- ★ To store container data, I generated two volumes, **vaishnavip_assignment3_clientvol**, and **vaishnavip_assignment3_servervol**.

```
exouser@assign3vvp:~/VaishnaviP_Assignment3$ docker volume ls
```

DRIVER	VOLUME NAME
local	clientvol
local	servervol
local	vaishnavip_assignment3_clientvol
local	vaishnavip_assignment3_servervol

```
exouser@assign3vvp:~/VaishnaviP_Assignment3$
```

Communication between the two:

- ★ Communication between containers occurs via client.py and server.py, as seen by file transfers over the network.
- ★ I have used **docker network ls** to list all the networks.
- ★ As shown in the screenshot below, I successfully constructed a network named **vaishnavip_assignment3_mydockernetwork**.


```
exouser@assign3vvp:~/VaishnaviP_Assignment3$ docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
ba50f8ea084d	bridge	bridge	local
79e8ee278b48	guacamole_exo-guac-net	bridge	local
16b638b4f904	host	host	local
9d90c106fd49	none	null	local
999d279f5182	vaishnavip_assignment3_mydockernetwork	bridge	local

- ★ Both containers, **vaishnavip_assignment3_server_1** and **vaishnavip_assignment3_client_1**, use the same network, as shown in the screenshot below.
- ★ All these have been configured in `docker-compose.yml`, as described in the File section above.
- ★ All the details of the network are shown using the **docker network inspect vaishnavip_assignment3_mydockernetwork** command.

```
exouser@assign3vvp:~/VaishnaviP_Assignment3$ docker network inspect vaishnavip_assignment3_mydockernetwork
```

```
[
  {
    "Name": "vaishnavip_assignment3_mydockernetwork",
    "Id": "999d279f5182d94d74fa381a9785a0f3e66dce256a59e88a3f2daef30ce0e8cf",
    "Created": "2024-04-25T05:47:24.974284501Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.25.0.0/16",
          "Gateway": "172.25.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": true,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {
      "88c9e3e73708ad862e45f20a80d5654153e5e4d6ab9fc1272fea4b60b06468b5": {
        "Name": "vaishnavip_assignment3_client_1",
        "EndpointID": "677823f7e688fb4cc3a2b1e5fba658fb6d0fc92ba12b138e7c8a9c8c2ddaa6fa",
        "MacAddress": "02:42:ac:19:00:03",
        "IPv4Address": "172.25.0.3/16",
        "IPv6Address": ""
      },
      "d56df328ddc62741f4e44c19bbe76ef7d06a664418c64ef9aec099be2f7db274": {
        "Name": "vaishnavip_assignment3_server_1",
        "EndpointID": "20ac6cbe9bc4ae5b6db87369ad2a72eb26efa494c800eb9c35947809b7a74529",
        "MacAddress": "02:42:ac:19:00:02",
        "IPv4Address": "172.25.0.2/16",
        "IPv6Address": ""
      }
    },
    "Options": {},
    "Labels": {
      "com.docker.compose.network": "mydockernetwork",
      "com.docker.compose.project": "vaishnavip_assignment3",
      "com.docker.compose.version": "1.29.2"
    }
  }
]
```

- ★ I examined the volumes to see if they had been formed correctly and to determine their location.
- ★ I used **docker volume inspect vaishnavip_assignment3_servervol** to inspect the servervol volume.

```
exouser@assign3vvp:~/VaishnaviP_Assignment3$ docker volume inspect vaishnavip_assignment3_servervol
[
  {
    "CreatedAt": "2024-04-24T05:45:14Z",
    "Driver": "local",
    "Labels": {
      "com.docker.compose.project": "vaishnavip_assignment3",
      "com.docker.compose.version": "1.29.2",
      "com.docker.compose.volume": "servervol"
    },
    "Mountpoint": "/var/lib/docker/volumes/vaishnavip_assignment3_servervol/_data",
    "Name": "vaishnavip_assignment3_servervol",
    "Options": null,
    "Scope": "local"
  }
]
```

- ★ I used **docker volume inspect vaishnavip_assignment3_clientvol** to inspect the clientvol volume.

```
exouser@assign3vvp:~/VaishnaviP_Assignment3$ docker volume inspect vaishnavip_assignment3_clientvol
[
  {
    "CreatedAt": "2024-04-24T05:45:14Z",
    "Driver": "local",
    "Labels": {
      "com.docker.compose.project": "vaishnavip_assignment3",
      "com.docker.compose.version": "1.29.2",
      "com.docker.compose.volume": "clientvol"
    },
    "Mountpoint": "/var/lib/docker/volumes/vaishnavip_assignment3_clientvol/_data",
    "Name": "vaishnavip_assignment3_clientvol",
    "Options": null,
    "Scope": "local"
  }
]
```

Important Note: I used the sudo commands to access volumes because the docker was already installed as the root user.

- ★ After launching the containers successfully, the server.py transfers **GeneratedFile.txt** to the client, and the client receives **FileReceived.txt**.
- ★ I verified whether the file was created by navigating to the volume location, which we obtained using the above inspection statement.
- ★ The snapshot shows that the file was successfully created.

```
exouser@assign3vvp:~/VaishnaviP_Assignment3$ sudo ls -l /var/lib/docker/volumes/vaishnavip_assignment3_servervol/_data
total 4
-rw-r--r-- 1 root root 1024 Apr 25 05:47 GeneratedFile.txt
exouser@assign3vvp:~/VaishnaviP_Assignment3$ sudo ls -l /var/lib/docker/volumes/vaishnavip_assignment3_clientvol/_data
total 4
-rw-r--r-- 1 root root 1024 Apr 25 05:47 FileReceived.txt
```

- ★ I also verified the cksums for both files, and they are identical, as seen below.

```
exouser@assign3vvp:~/VaishnaviP_Assignment3$ sudo docker-compose exec server cksum /serverdata/GeneratedFile.txt
1840097169 1024 /serverdata/GeneratedFile.txt
exouser@assign3vvp:~/VaishnaviP_Assignment3$ sudo docker-compose exec client cksum /clientdata/FileReceived.txt
1840097169 1024 /clientdata/FileReceived.txt
```

Q. You should be able to get into the shell of the client container to physically check if the file has been received.

- ★ As stated in the assignment, I am also sending and checking the checksum in client.py.
- ★ The client container shell screenshot is shown below.
- ★ To open the client container shell, I used **docker-compose exec client sh**.
- ★ I went to /clientdata to confirm that the file had been received.
- ★ The file was successfully received, and the checksum is accurate.

```
exouser@assign3vvp:~/VaishnaviP_Assignment3$ docker-compose exec client sh
#
#
# pwd
/app
# cd ../
# ls -lart
total 64
drwxr-xr-x 2 root root 4096 Jan 28 21:20 home
drwxr-xr-x 2 root root 4096 Jan 28 21:20 boot
drwxr-xr-x 1 root root 4096 Apr 8 00:00 var
drwxr-xr-x 1 root root 4096 Apr 8 00:00 usr
drwxr-xr-x 2 root root 4096 Apr 8 00:00 srv
lrwxrwxrwx 1 root root 8 Apr 8 00:00 sbin -> usr/sbin
drwxr-xr-x 2 root root 4096 Apr 8 00:00 opt
drwxr-xr-x 2 root root 4096 Apr 8 00:00 mnt
drwxr-xr-x 2 root root 4096 Apr 8 00:00 media
lrwxrwxrwx 1 root root 9 Apr 8 00:00 lib64 -> usr/lib64
lrwxrwxrwx 1 root root 7 Apr 8 00:00 lib -> usr/lib
lrwxrwxrwx 1 root root 7 Apr 8 00:00 bin -> usr/bin
drwxr-xr-x 1 root root 4096 Apr 10 05:25 run
drwx----- 1 root root 4096 Apr 24 04:12 root
drwxrwxrwt 1 root root 4096 Apr 24 04:12 tmp
drwxr-xr-x 2 root root 4096 Apr 25 05:09 clientdata
drwxr-xr-x 1 root root 4096 Apr 25 05:25 app
drwxr-xr-x 1 root root 4096 Apr 25 05:47 etc
-rwxr-xr-x 1 root root 0 Apr 25 05:47 .dockerenv
dr-xr-xr-x 13 root root 0 Apr 25 05:47 sys
dr-xr-xr-x 377 root root 0 Apr 25 05:47 proc
drwxr-xr-x 5 root root 340 Apr 25 05:47 dev
drwxr-xr-x 1 root root 4096 Apr 25 05:47 ..
drwxr-xr-x 1 root root 4096 Apr 25 05:47 .
# cd clientdata
#
# ls -lart
total 12
drwxr-xr-x 2 root root 4096 Apr 25 05:09 .
drwxr-xr-x 1 root root 4096 Apr 25 05:47 ..
-rw-r--r-- 1 root root 1024 Apr 25 05:47 FileReceived.txt
# cat FileReceived.txt
7dAHE0UvnmBgW8HRBKKVUT2gEC1F78VtV6ZJ2NnXhrR4ELiIrr0UykEXLh6MADgmoELCf2vYekEIy8FcTcsUCk0at0E7zPYIfNzxfvU1DDIyY6WKUsM8ZvhR3KB0SLWA
RPdHQVxbnDu8tUubQccGNm5KqWiz0eSr3qLLSo3qo4JhSHCCa5s3G1i0pMQUs2bb6WccX9DM3luXyTbK2Aq6DwIRfXE073llantfMiv3CxxhSLYgl90mcjcEofhKd
nu38wzT21NZxxrFVutFEer0lFxAU7RtEgJdWosyqFtJ5s1zpSP50zN0pwrXnnJje3y95Hfx1be0cSrB8IeZmuMp007sYHZZpJz1FPnbwKICitPGr2eRLJyh0lRPkhW
cWxoIaIZmxYPpm7yPCLh61qFPK0lJM4pHzE5MgncISEX1GBs2R3vJFPrbTVSf0LTl9ijTsI4Ax3oZxRf4icLFdhDYQP24h62Jz0n38sZ3cleBw0A08HrikqyFd0z0sa
157pGeIDW79cpVpcXuGhap0cNNZUYs2G9IzbyjD6Bm976Sween2FJ0s2UIA20YBhC4CWNbv2rGfrCTM00uM1caxLaFHYrWuDKWASpLxL8rwyxP1Tltqb1N4lgJkaL
2s6oV7ugdCKXg6f7Q6uLndgLeIss3564DBauUe2Rg9faY3Stjbbv65BymPs1VMTxwrnq28fxRD0MGK1xVCRGPiobm1WzM6z74Cf15GuurJhZMCzJrd2v1R8c3XZSRth
h4e53rUTradiBbqXgSuxxou3ym0iHxcV6ch00TJmlB6saUVytIfunaFRLCSd7WviScetGwk7XDDBaoUppXAKCSiMMdeivOWkD8SXs06Phd1UspUHuXy9Tj0fnBI87T
hrJQDwgBwq9wVxvoFxmxc8BHBGcRvIV90LKPEVgI3va0vUwALSAH20s3J18CPncDXV0fz9FE0fDMgKYC5D2z75xAWjvWQ8APsks9SetU9mBk38XZ4FL4qFMS1DBCwP
tpq2r1sf#
#
#
# cksum FileReceived.txt
1840097169 1024 FileReceived.txt
# exit
```